

GraphHist: Reproduction and a Proposed Correction

Graph Self-Supervised Learning for Histopathology

Aadith Krishna CB.SC.U4AIE23202

Aditya Santosh CB.SC.U4AIE23207

Arjungopal Anilkumar CB.SC.U4AIE23271

Group 1

Introduction: why histopathology is hard for deep learning

A pathology slide is scanned into a **Whole Slide Image (WSI)**, roughly $100,000 \times 80,000$ pixels and 1–3 GB per slide.

The standard approach

Cut the slide into small tiles, push each tile through a vision model such as a CNN or a Vision Transformer, then pool the tile features together.

Why it is unsatisfying

A vision transformer sees a *fixed grid* of square patches. Real tissue is made of *cells* at irregular positions. The grid does not line up with biology.

The biological intuition

What matters diagnostically is *which cells are present and how they are arranged relative to each other*, not the raw pixels. That is naturally a **graph**, not an image.

Problem statement

The problem the paper sets out to solve

Vision models applied to histopathology are expensive to train, operate on a fixed grid that ignores cellular structure, and scale *quadratically* with the number of image tokens.

Can tissue instead be represented as a graph of cells, so that a model learns cellular organisation directly, and does so at a much lower computational cost?

Why this is a hard problem

- Labels are scarce. Expert annotation of pathology slides is slow and expensive, so the model must learn *without labels*.
- Tumour tissue is *heterogeneous*. Neighbouring cells are often of different types, which breaks the assumption most graph networks are built on.
- A single slide can contain millions of cells, so the method has to stay cheap at scale.

GrapHist: Graph Self-Supervised Learning for Histopathology

Ögüt et al., arXiv:2603.00143

The thesis

Represent tissue as a *cell graph*, pre-train on it without labels, and you obtain better representations than vision models at a fraction of the cost.

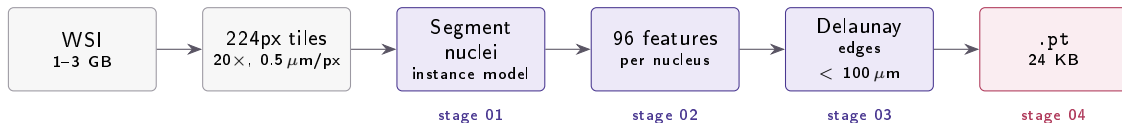
Headline claims

- Beats vision self-supervised models at slide, region and cell level
- **9.50M parameters**, roughly **4× fewer** than the vision models it is compared against
- Far less pre-training compute
- Works from small 224px tiles up to whole regions

Scale of the work

- Pre-trained on **11,149,500 cell graphs**
- Derived from 998 breast cancer whole slide images
- Evaluated on 5 downstream datasets
- 89 published result values in total

Methodology 1: turning a slide into cell graphs



- Tissue is separated from background, rescaled to 20 $\times$ magnification (0.5 μm per pixel), then cut into non-overlapping 224×224 tiles.
- A **nuclei instance segmentation model** detects every individual nucleus. The surrounding tissue is discarded, leaving a cell-centric representation.
- Each nucleus becomes a **node** carrying 96 descriptors. **Delaunay triangulation** connects neighbouring cells, and edges longer than 100 μm are cut as biologically implausible.
- Each edge is weighted by the distance between the two cells, in microns.

Methodology 2: what a .pt file actually contains

.pt is simply PyTorch's save format. In GrapHist, each one holds **one cell graph for one tile**:

```
Data(x=[31, 96], edge_index=[2, 81], edge_attr=[81, 1],  
      sample_id='Adrenal_gland_1_01171', labels=[31])      # 16,320 bytes
```

Field	Shape	Meaning
x	[31, 96]	31 nuclei, each described by 96 numbers
edge_index	[2, 81]	81 connections between nuclei
edge_attr	[81, 1]	length of each connection, in microns
labels	[31]	the cell type of each nucleus

The key point: the transformation is one way

No pixels survive. A nucleus that was hundreds of pixels of stained image is now 96 numbers. One slide becomes about 11,000 .pt files of roughly 24 KB each. Because the authors published these finished files, we did not have to rebuild them.

Methodology 3: the architecture, handling mixed neighbourhoods

The problem a normal graph network has here

Standard graph networks *average* each cell with its neighbours, which assumes neighbours are similar. In a tumour a cancer cell sits beside an immune cell, so averaging destroys the very contrast a pathologist looks for.

GrapHist uses **Adaptive Channel Mixing (ACM)**. Every layer passes the graph through **three channels at once**:

Channel	Operator	What it does
Low-pass	$A_L = (I + A_{rw})/2$	blends a cell with its neighbours, smoothing uniform tissue
High-pass	$A_H = (I - A_{rw})/2$	takes the <i>difference</i> from neighbours, sharpening boundaries
Identity	$A_I = I$	keeps the cell's own features untouched

What makes it “adaptive”

For **each cell separately**, the network scores all three channels and takes a *learned* weighted blend. A cell on a tumour boundary can lean on the high-pass channel while a cell inside uniform tissue leans on the low-pass one. Two extras help: a **virtual node** joined to every cell for long-range context, and **jumping knowledge**, which combines all layers.

Methodology 4: self-supervised pre-training

No labels are used at this stage. The model learns by **hiding parts of the graph and reconstructing them**.

- 1 Randomly hide the features of a subset of cells in the graph.
- 2 An **encoder** reads the corrupted graph and produces an embedding for every cell.
- 3 The hidden cells are masked again inside that embedding.
- 4 A **decoder** tries to reconstruct their original 96 features.
- 5 The model is rewarded for reconstructing them accurately.

Why this works

To guess a hidden cell's features, the model has no choice but to read the cells around it. It therefore learns *cellular context* on its own, from 11.1 million graphs, without a single human label.

Outcome: a **9.50M parameter** encoder, pre-trained once and released as `graphist.pt`. This is the checkpoint we evaluate.

Methodology 5: how the model is evaluated

The encoder is **frozen**. It is never fine-tuned. Only a small classifier on top is trained, which tests the *quality of the representation itself*.

Scale	How the embedding is built	Classifier
Cell	the cell embedding, used directly	logistic regression
Region	average of all cell embeddings in the region	logistic regression
Slide	attention pooling over region embeddings	attention MIL

Why freeze the encoder?

If the encoder were fine-tuned, a good score could come from the fine-tuning rather than from the pre-training. Freezing it means the score reflects only what self-supervised pre-training actually learned.

Understanding the metrics: what each number means

All four are reported as percentages, and higher is better for all four.

Metric	What it measures
Macro F1	Balance of precision and recall, computed <i>separately for each cell type</i> and then averaged with equal weight, so a model that ignores rare cell types is heavily punished. This is the headline metric.
Balanced accuracy	The average, across classes, of “what fraction of this class did we correctly find?”. Random guessing scores $1/k$ for k classes, so 20% for PanNuke’s 5 classes.
AUROC	The probability the model ranks a random positive example above a random negative one. 50 is random and 100 is perfect. It measures <i>ranking quality</i> rather than decisions.
AUPRC	Area under the precision–recall curve. Unlike AUROC it ignores true negatives, making it the more honest metric when a class is rare, which is the normal situation in cell typing.

△ on the next slide is the absolute gap between our value and the paper’s, in percentage points. Smaller is better.

Results: cell level, ours vs the paper

Reproduced using **only what the authors released**: the pre-trained checkpoint, the published datasets and the public code.

Setting	Metric	Ours	Paper	Δ
PanNuke PanCancer	Macro F1	59.54 ± 0.08	59.47 ± 0.11	0.07
	Balanced acc	68.70 ± 0.98	68.73 ± 0.96	0.03
	AUROC	89.68 ± 0.04	89.57 ± 0.05	0.11
	AUPRC	66.45 ± 0.85	66.30 ± 0.94	0.15
PanNuke Breast	Macro F1	56.39 ± 0.06	56.43 ± 0.04	0.04
	Balanced acc	57.49 ± 0.07	57.49 ± 0.07	0.00
	AUROC	85.49 ± 8.13	85.30 ± 8.07	0.19
	AUPRC	61.94 ± 0.27	61.85 ± 0.30	0.09
NuCLS main	Macro F1	25.96 ± 2.51	26.57 ± 2.16	0.61
	AUROC	72.41 ± 1.68	71.49 ± 1.55	0.92
NuCLS super	Macro F1	46.52 ± 2.53	46.24 ± 3.89	0.28
	Balanced acc	50.96 ± 5.28	48.04 ± 3.14	2.92

PanNuke reproduces: eight metrics, largest deviation 0.19 points, one exact match. The odd ± 8.07 spread on Breast AUROC looks like a typo, but it reproduces as ± 8.13 (one weak fold: 73.99 against 91.56 and 90.91). **Reproducing an anomaly of that shape is stronger evidence than a clean average.**

Region level

Dataset	Ours	Paper	Δ
BreakHis	90.43	89.37	+1.06
BRACS	62.26	60.30	+1.96

Macro F1. Both land *within two points* of the paper's pipeline while using one graph for the whole region instead of splitting it into tiles. This supports the paper's claim that performance stays stable across patch sizes.

Structural checks, all verified

- Parameter counts: exact at all three model sizes
- 96 node features: confirmed by tracing the extraction code
- Graph statistics: match to two decimals
- Class distributions: all 16 counts exact

What this establishes

The published model behaves exactly as reported. That matters for the next part: it gives us a *trustworthy baseline* to measure any proposed change against.

Research gap: the virtual node is applied in the wrong space

Recall the **virtual node**, the extra node joined to every cell to carry long-range context. Its connection strength has to be set to some value.

What the paper says

Its edge weights are set to *“the mean of all edge weights in the pre-training dataset”*.

What the code does

Uses a fixed value of 22.5, and inserts it *after* all the other edge weights have already been rescaled.

After rescaling, the average edge weight is 18.627 with a spread of 12.735, so the correct value on that scale is $(22.5 - 18.627)/12.735 = 0.304$.

The inserted value is **roughly 74 times too large**.
Converted back, 22.5 corresponds to about $305\ \mu\text{m}$, which is three times the $100\ \mu\text{m}$ biological cutoff the graphs themselves enforce.

Research gap: what we measured

Measured on 300 real PanNuke graphs.

Configuration	bad degrees	neighbour vs self	low vs high similarity
As shipped, value 22.5	0.0%	0.080	0.992
Simple fix, value 0.304	57.8%	1.009	-0.012
Distance kernel (our proposal)	0.0%	0.787	0.320

Finding A: the headline mechanism is inert

The virtual node absorbs **99.92%** of everything each cell aggregates. As a result the low-pass and high-pass channels come out **99.2% identical**. *The adaptive channel mixing has nothing left to mix*, so the mechanism the paper's argument rests on is not actually operating.

Finding B: the real issue is where the value is applied

Rescaled weights can be *negative*, but the averaging step requires positive ones: **56% of cells have a negative total**. The 22.5 value was accidentally hiding this, so simply correcting it to 0.304 makes things *worse*. A positive distance-based weighting fixes both problems at once.

Honest positioning

The authors' own dataset notes call distance-based weights “counterintuitive” and suggest trying the inverse of distance, so **the idea of a different weighting is not ours**. But they never measured it, every published number uses the original weighting, and their notes say nothing about the virtual node, the 99.92% share, or the negative totals. *That interaction is our contribution.*

Proposed experiment: three full pre-training runs on identical data, epochs and seed, changing only the edge weighting.

Arm	Edge weight	Virtual node	Purpose
A	rescaled distance (as shipped)	22.5	reproduce the checkpoint, validate our pipeline
B	inverse of distance	matched mean	test the authors' own unmeasured suggestion
C	distance kernel, corrected	matched mean	our proposed correction

Feasible: the full 11.1 million graph pre-training corpus *is* published, with official splits, and an A100 is available at roughly two days per arm. **Prediction:** the correction should help most where mixed neighbourhoods matter, that is, boundary cell types in NuCLS and PanNuke, measured against the baseline our reproduction established.

Conclusion

The published results hold

All eight PanNuke metrics reproduce within 0.19 percentage points, the region-level results within two points, and every structural claim checks out exactly. **GraphHist's reported performance is real.**

But the mechanism behind it is not the one described

The virtual node absorbs **99.92%** of everything each cell aggregates, leaving the low-pass and high-pass channels **99.2% identical**. The adaptive channel mixing that the paper's whole argument rests on is not actually operating.

The cause is precise and the fix is testable

The value is applied on the wrong scale, and correcting it naively makes things worse. A positive distance-based weighting repairs both the scale and the negative-total problem at once.

Next step: three controlled pre-training runs on the published corpus, to measure what the correction is actually worth.

Backup: why do our numbers differ from the paper's?

First, the premise

We did not retrain anything. The encoder is the authors' released checkpoint, frozen. There is no random initialisation and no seed in our pipeline, so training randomness cannot explain any of this.

Result family	Δ	Actual cause
PanNuke	0.00 to 0.19	floating-point noise: A100 GPU vs Apple CPU, and newer libraries
NuCLS	0.28 to 3.05	the official fold splits were never published, so we rebuilt them
Region level	+1.06, +1.96	different input: one whole-region graph instead of patches

PanNuke. Float32 sums accumulate in a different *order* on a GPU than on a CPU. Those $1e-6$ differences pass through 5 layers and then hit a linear decision boundary, so a few cells out of 163,163 flip class. **This is the floor for cross-platform reproduction.**

NuCLS. A different fold assignment is a different experiment, so the numbers *should* move. The defence is that our gaps are smaller than the paper's own fold-to-fold spread: Macro F1 0.28 vs ± 3.89 , balanced accuracy 2.92 vs ± 3.14 , AUROC 1.16 vs ± 2.08 , AUPRC 3.05 vs ± 2.59 .

Evidence it is noise, not error. The odd ± 8.07 spread on Breast AUROC is caused by one weak fold (73.99 against 91.56 and 90.91). We reproduced it as ± 8.13 . **A real methodological difference would smear that artefact out, not reproduce its exact shape.**